

# Reinforcement Learning for a Staged Pick-and-Place Robotic Arm in Unity ML-Agents

Mengyuan Pan

University of the Arts London  
London, United Kingdom  
m.pan0220251@arts.ac.uk

## Abstract

This project investigates the use of reinforcement learning for a simulated robotic Pick-and-Place task in Unity ML-Agents. The objective is to train a robotic arm to approach a target object, attach or grasp it, transport it to a placement zone, and release it safely. Since the complete task involves a long sequence of dependent behaviours, direct end-to-end training was difficult and unstable. Therefore, the system was developed through a staged curriculum learning strategy consisting of four phases: Reach, Attach, Transport, and Lower-and-Release. The agent was trained using Proximal Policy Optimisation (PPO), with six continuous actions controlling the robotic arm joints. The observation vector was kept at 20 dimensions across the later stages so that policies could be initialised from earlier trained models. The reward function was redesigned for each stage to guide the agent from simple reaching towards the full Pick-and-Place pipeline. The results show that the staged strategy produced positive reward trends in all phases and enabled the robot to perform the main task sequence in simulation. However, limitations remain in physical grasp realism, transport stability, and release precision.

## CCS Concepts

• **Computing methodologies** → **Reinforcement learning; Robotics**; • **Hardware** → *Robotic control*; • **Applied computing** → *Simulation*.

## Keywords

Reinforcement Learning, Pick and Place, Unity ML-Agents, PPO, Robotic Arm, Robotic Manipulation, Curriculum Learning, Simulation

## 1 Introduction

Pick-and-Place is a fundamental robotic manipulation task widely used in industrial automation, logistics, and robotic simulation. A complete Pick-and-Place pipeline usually requires a robot to identify the target object, move the end-effector towards it, grasp or attach the object, transport it to a desired location, and release it accurately. This project focuses on the technical formulation of this task as a reinforcement learning problem in a Unity-based simulation environment.

Traditional robotic control methods often depend on manually designed trajectories and carefully tuned control rules. Reinforcement learning (RL), in contrast, allows an agent to learn action policies through repeated interaction with an environment. This makes RL suitable for continuous robotic control tasks, especially when the robot must learn a sequence of behaviours rather than execute a fixed pre-programmed motion.

The goal of this project is to train a simulated robotic arm to complete a staged Pick-and-Place task. The emphasis is not on achieving industrial-level grasping accuracy, but on exploring how the task can be represented through observations, actions, and rewards. The project therefore investigates how a robotic manipulation problem can be decomposed into learnable RL stages and how the reward function can be iteratively refined according to observed training failures.

At the beginning of development, training the full task directly was found to be difficult because the reward was sparse and the required action sequence was long. The agent had to learn reaching, grasping, transporting, lowering, and releasing simultaneously. To make the learning process more manageable, the task was divided into four stages: Reach, Attach, Transport, and Lower-and-Release.

## 2 Background

### 2.1 Reinforcement Learning for Robotic Control

In reinforcement learning, an agent observes the current state of the environment, selects an action according to a policy, and receives a reward. The objective is to learn a policy that maximises expected cumulative reward over time [1]. In this project, the robot state is represented by joint angles, end-effector position, target-object position, placement-zone position, and task-related information such as whether the target is attached. The action space consists of continuous control signals applied to the robotic arm joints. The reward function encourages the agent to approach the target, attach it, carry it to the placement area, and release it safely.

Robotic manipulation is a challenging RL problem because it involves continuous actions, contact-rich dynamics, and long-horizon dependencies. Small errors in early stages can affect later behaviours. For this reason, staged or curriculum-based training is useful in this project, because it allows the agent to learn simpler behaviours before attempting the full task. This follows the broader observation in robot reinforcement learning that task representation, prior structure, and careful problem formulation are important for handling the complexity of robotic control [2].

### 2.2 Proximal Policy Optimisation

This project uses Proximal Policy Optimisation (PPO), as implemented in Unity ML-Agents. PPO is a policy-gradient method designed to improve training stability by limiting the size of policy updates [3]. This is useful for continuous control tasks such as robotic arm movement, where excessively large policy updates may lead to unstable or unsafe actions.

## 2.3 Unity ML-Agents

Unity ML-Agents provides a framework for connecting Unity simulation environments with machine learning training pipelines [4, 5]. In this project, Unity was used to construct the robotic arm scene, target object, and placement zone, while ML-Agents was used to collect observations, apply actions, calculate rewards, and train the PPO policy.

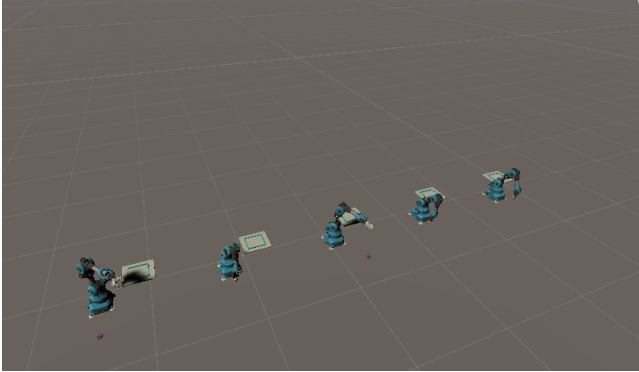
## 2.4 Pick-and-Place Formulation

A Pick-and-Place task can be divided into perception, approach, grasp, transport, and placement stages, which also matches the staged logic used in Unity Robotics Hub Pick-and-Place examples [6]. This project does not implement visual perception. Instead, it uses positions and relative vectors from the Unity environment as observations. To reduce the instability of physical grasping, a simplified magnetic attachment mechanism was used. When the grasp point moved close enough to the target, the target was attached to the grasp point and moved with the gripper. This design allowed the project to focus on RL-based arm control rather than detailed contact physics.

## 3 Technical Implementation

### 3.1 Simulation Environment

The simulation environment was built in Unity and contains a Niryo six-joint robotic arm, a target object, a safe placement zone, a gripper controller, an ML-Agents agent script, and a PPO training configuration. The robot model was set up as an articulated URDF-based arm in Unity, with the main links exposed as shoulder\_link, arm\_link, elbow\_link, forearm\_link, wrist\_link, and hand\_link. The robotic arm joints were controlled through Unity’s ArticulationBody system, consistent with the Unity robotics workflow for imported articulated robots [7]. Each joint has an xDrive target, and the policy changes these joint targets through continuous action values.



**Figure 1: Unity simulation environment showing multiple training instances, the robotic arm, target object, and placement zone.**

The target object and placement zone were positioned in a constrained workspace. In the early reaching stage, the target was randomised within a square area around spawnCenter. In the later transport and release stages, the Safeplacement zone was also

**Table 1: Main implementation files.**

| File                       | Role in the project                                                                                                                                            |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NiryoReachAgent.cs         | Step 1 reaching agent. It trains the end-effector or graspPoint to move towards a randomly spawned target.                                                     |
| STEP2_Grasp.cs             | Step 2 approach and attachment stage. It keeps the observation size compatible with Step 1 and adds scripted gripper closure and magnetic attachment.          |
| STEP3_Transport.cs         | Step 3 transport stage. It keeps the Step 2 attach behaviour and changes the active goal to a point above the safe placement zone once the object is attached. |
| Step4_Release.cs           | Step 4 full placement stage. It adds lowering, safe release, and final placement validation.                                                                   |
| SimpleGripperController.cs | Scripted gripper helper for opening and closing the fingers.                                                                                                   |

randomised and used as the active transport or release goal. This randomisation helped prevent the policy from only memorising a single fixed position.

### 3.2 Implementation Files and Training Setup

The implementation was separated into stage-specific C# scripts. This made it easier to debug each phase without changing the whole pipeline at once. The final report is based on the scripts listed in Table 1. All stages used the same behaviour name, NiryoReach, and the learned policy used a 20-dimensional vector observation space with 6 continuous actions in the later stages.

The Unity project used four YAML training configurations. To avoid confusion with the actual project folders, the training configuration and output folder for each phase are shown below.

- Phase 1: ppo\_phase1\_reach.yaml  
output folder: phase1\_v3
- Phase 2: step2\_attach.yaml  
output folder: phase2\_attach\_from\_phase1\_move03
- Phase 3: ppo\_phase3\_lift.yaml  
output folder: p3\_reach\_to\_zone\_v1
- Phase 4: ppo\_phase4\_place.yaml  
output folder: p4\_release\_safe\_snap\_v6

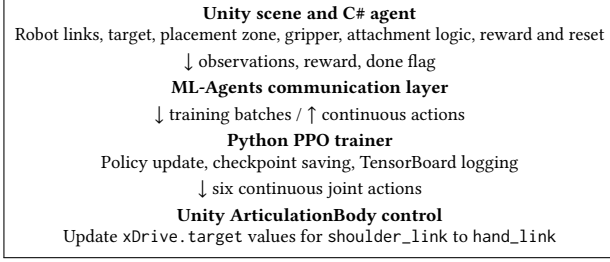
These names were kept in the result tables so that the report directly matches the trained models and TensorBoard runs.

### 3.3 System Architecture and Module Communication

The system was organised as a closed training loop between the Unity simulation and the ML-Agents Python trainer. Unity was responsible for the physical scene, robot state updates, target and placement-zone reset logic, scripted gripper behaviour, magnetic attachment, reward calculation, and episode termination. The Python trainer was responsible for running PPO, updating the neural network policy, saving checkpoints, and writing TensorBoard logs.

At every decision step, the active C# agent script collected a 20-dimensional vector observation from the Unity scene. This observation included the robot joint state, grasp-point position, active goal position, relative vectors, attachment state, height information, and episode progress. The observation, the reward from the previous step, and the episode status were sent from Unity to the ML-Agents trainer. The trainer then used the current PPO policy

to output six continuous action values. Unity received these actions, converted them into incremental joint target updates for the six ArticulationBody links, advanced the physics simulation, recalculated reward, and repeated the loop.



**Figure 2: System architecture and data flow between Unity, the ML-Agents trainer, and the robotic arm controller.**

The four training stages shared the same behaviour name, *NiryoReach*, and the later-stage scripts preserved the same 20-dimensional observation size and 6-dimensional continuous action size. This was important because it allowed the neural network weights from one phase to initialise the next phase. In practice, Phase 1 was trained from scratch. Phase 2 reused the Phase 1 reaching policy as its starting point, Phase 3 reused the Phase 2 attachment policy, and Phase 4 reused the Phase 3 transport policy. The C# scripts did not directly pass data to one another during runtime; instead, the connection between stages was made through the saved ML-Agents policy checkpoints and the consistent observation-action interface.

### 3.4 Robot and Gripper Control

The robot has six controllable Unity ArticulationBody links assigned in the Inspector: *shoulder\_link*, *arm\_link*, *elbow\_link*, *forearm\_link*, *wrist\_link*, and *hand\_link*. In the C# scripts these are exposed as *jointA1* to *jointA6*, but the report uses the actual Unity link names to match the scene setup. The PPO policy outputs six continuous values, each corresponding to one of these links. These actions are interpreted as incremental changes to the joint target angles and are clamped within the safe joint range from -120 degrees to 120 degrees. In the later stages, the raw actions were also smoothed and scaled. This reduced jitter and prevented the arm from making overly aggressive movements near the target or placement zone.

The gripper was controlled using a separate scripted controller rather than being fully learned by the agent. This simplified the task and reduced the difficulty of physical grasping. In the attachment stage, when the distance between the grasp point and the target object became smaller than a defined threshold, the object was magnetically attached to the grasp point and the gripper was closed. This design made training more stable, although it also reduced the realism of the grasping process.

### 3.5 Observation Space

The observation space combines robot state, object state, goal state, and task progress information. In Step 1 and Step 2, the goal is mainly the target object. In Step 3 and Step 4, the active goal changes

**Table 2: Main observation components used by the agent.**

| Observation                            | Purpose                                                           |
|----------------------------------------|-------------------------------------------------------------------|
| Normalised joint angles                | Represent the current robot posture.                              |
| Grasp-point position                   | Locate the end-effector in the workspace.                         |
| Target-object or active-goal position  | Guide reaching, transport, and lowering.                          |
| Vector from grasp point to active goal | Provide directional information for the current sub-task.         |
| Distance to active goal                | Give a scalar progress signal.                                    |
| Grasp-point height                     | Help control lifting and lowering.                                |
| Attachment / gripper state             | Distinguish approaching, carrying, lowering, and released states. |
| Best distance and episode progress     | Support progress tracking and time-limited behaviour.             |

**Table 3: Continuous action space.**

| Action   | Description                                      |
|----------|--------------------------------------------------|
| Action 1 | Controls <i>shoulder_link</i> / <i>jointA1</i> . |
| Action 2 | Controls <i>arm_link</i> / <i>jointA2</i> .      |
| Action 3 | Controls <i>elbow_link</i> / <i>jointA3</i> .    |
| Action 4 | Controls <i>forearm_link</i> / <i>jointA4</i> .  |
| Action 5 | Controls <i>wrist_link</i> / <i>jointA5</i> .    |
| Action 6 | Controls <i>hand_link</i> / <i>jointA6</i> .     |

depending on the stage: the object while approaching, the placement point while carrying, and the lower release point during the final placement stage. The vector observation size was kept at 20 dimensions so that later models could be initialised from earlier trained policies.

### 3.6 Action Space

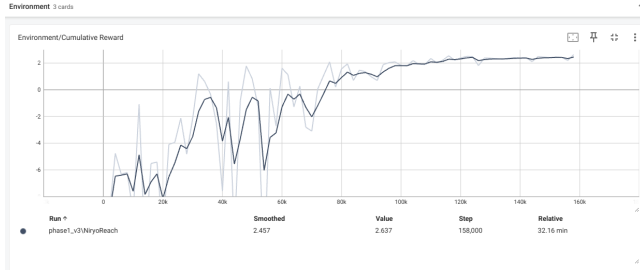
The project uses a continuous action space. Each action value controls one robotic arm joint, as shown in Table 3. Continuous actions are appropriate for this task because robotic arm control requires smooth adjustment of joint angles rather than discrete movement commands.

### 3.7 Reward Design

The reward function was not designed all at once. Instead, it was iteratively adjusted after observing failure cases in Unity and in TensorBoard. Each stage used a different reward emphasis, because each stage had a different learning objective.

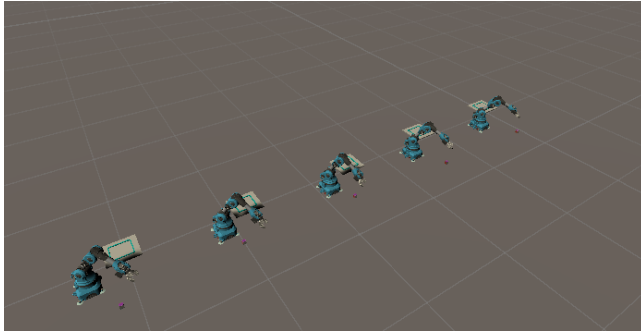
**3.7.1 Step 1: Reach Reward.** The first stage trained the end-effector to approach the target object. The target was randomly spawned within a defined square area. The reward was mainly based on the change in distance between the grasp point and the target. If the distance became smaller, the agent received positive reward; if

the distance increased or the agent stopped improving, it received penalties. A success reward was given when the grasp point entered the target threshold. This stage also helped establish the basic joint-control logic for later training.

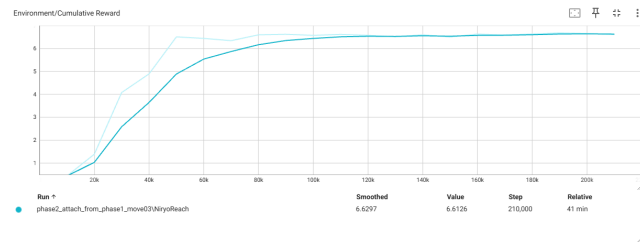


**Figure 3: Phase 1 cumulative reward curve for reaching. The smoothed reward increased to 2.457 at 158k steps.**

**3.7.2 Step 2: Attach Reward.** The second stage extended reaching into grasping and attachment. The agent first had to approach the target. When the grasp point entered the attachment range and the scripted gripper had closed sufficiently, the target object was magnetically attached to the grasp point. The reward included approach improvement, close-zone reward, attach-ready reward, attachment bonus, stable hold reward, and penalties for unsafe movement or lack of progress. One key difficulty was setting the correct distance threshold: it had to be reachable for the robotic arm but not so large that the attachment became unrealistic.

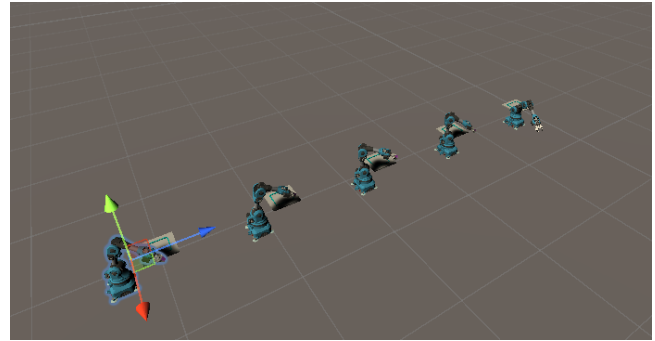


**Figure 4: Phase 2 behaviour example in Unity, where the robot learns to approach and attach the target object.**

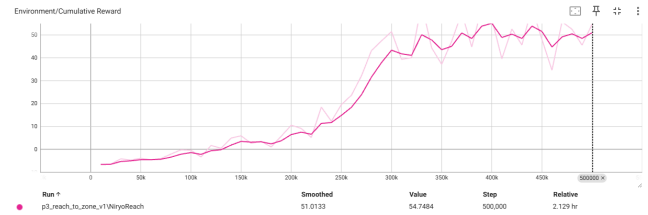


**Figure 5: Phase 2 cumulative reward curve. The smoothed reward reached 6.6297 at 210k steps.**

**3.7.3 Step 3: Transport Reward.** The third stage was the most challenging part of training. The agent had to keep the target locked to the grasp point while moving it towards a placement goal above the safe placement zone. In the code, the active goal switches from the target object to the placement goal after attachment. The reward encouraged the target to move closer to the placement zone, remain locked to the grasp point, and stay lifted above the spawn plane rather than being dragged. Instead of using a separate detachment event, the implementation keeps the object constrained through `KeepTargetLockedToGraspPoint()` and penalises the lock-position error between the object and the grasp point. Additional penalties were applied for low carrying height, lack of progress, high action changes, high average joint velocity, and unsafe states detected through workspace-distance, height, and joint-velocity limits.



**Figure 6: Phase 3 transport behaviour example. The agent carries the attached target towards the placement zone.**



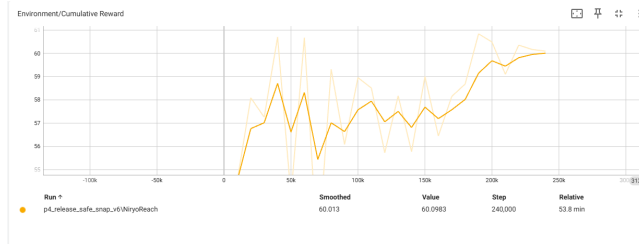
**Figure 7: Phase 3 cumulative reward curve. The smoothed reward increased to 51.0133 at 500k steps, but visible fluctuations remained.**

**3.7.4 Step 4: Lower-and-Release Reward.** The final stage trained the robot to lower the target object and release it safely inside the placement zone. This script keeps the approach, attach, and transport behaviour, then adds a lowering stage and a safe snap release. Instead of immediately enabling normal object physics after release, the released object remains kinematic and is held at a calculated safe placement pose while the script checks whether it stays within the allowed placement tolerance. This avoids a correct placement being ruined by sudden gravity, contact forces, or finger collisions.

**Table 4: Four-phase curriculum learning strategy.**

| Phase | Task      | Initialisation | Main goal         |
|-------|-----------|----------------|-------------------|
| 1     | Reach     | From scratch   | Move to target    |
| 2     | Attach    | From Phase 1   | Attach target     |
| 3     | Transport | From Phase 2   | Carry to zone     |
| 4     | Release   | From Phase 3   | Lower and release |

The reward encouraged the target to remain close to the zone centre, descend to a safe release height, and be released only when the object was positioned correctly. Early versions often released the object too high, making it look as if the target had been thrown rather than placed. Later reward adjustments increased the importance of lowering, centring, stable release, and final in-zone validation.

**Figure 8: Phase 4 Unity example showing the robot lowering and releasing the object near the placement zone.****Figure 9: Phase 4 cumulative reward curve. The smoothed reward reached 60.013 at 240k steps.**

## 4 Staged Training Strategy

The full Pick-and-Place task was not trained from scratch as a single end-to-end problem. Instead, the project used staged curriculum training. Each stage focused on one sub-task, and later stages were initialised from the policy trained in the previous stage. This reduced training difficulty and made reward design easier to debug.

**Table 5: Training runs and final smoothed reward values.**

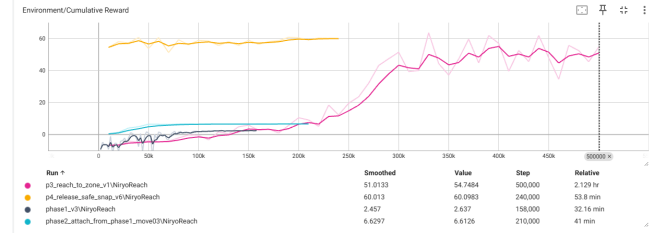
| Phase | Run name                         | Steps   | Reward  |
|-------|----------------------------------|---------|---------|
| 1     | phase1_v3                        | 158,000 | 2.457   |
| 2     | phase2_attach_from_phase1_move03 | 210,000 | 6.6297  |
| 3     | p3_reach_to_zone_v1              | 500,000 | 51.0133 |
| 4     | p4_release_safe_snap_v6          | 240,000 | 60.013  |

Note. Reward values are not directly comparable across phases because each phase used a different reward function.

## 5 Results

### 5.1 Reward Curves

The TensorBoard cumulative reward curves show positive learning trends across all four stages. Phase 1 began with unstable negative rewards, but the reward became positive after the agent learned a basic reaching behaviour. Phase 2 increased more quickly, which suggests that initialising from the Phase 1 reaching policy helped the agent learn attachment. Phase 3 showed a slow start followed by a sharp improvement after approximately 220k steps, indicating that transport behaviour gradually emerged. Phase 4 started from a relatively high reward level because it was based on the previous transport policy and then improved further as the release strategy became more stable.

**Figure 10: Combined TensorBoard cumulative reward curves for the four training phases. Each phase shows an increasing reward trend, supporting the use of staged training.**

It is important to note that the absolute reward values should not be compared directly across phases because the reward functions were redesigned for different objectives. The more meaningful observation is that each phase shows an internal upward trend, suggesting that the corresponding behaviour was being learned.

### 5.2 Behaviour Evaluation

In Phase 1, the robotic arm learned to move the end-effector towards the target object. Early training often produced large or unstable movements, but the final policy reached the target more consistently.

In Phase 2, the agent learned to approach the target and trigger the attachment mechanism. The simplified magnetic attachment made the grasping process more stable than relying on pure physical collision and friction. However, it also means that the grasping behaviour is only an approximation of real robotic grasping.

In Phase 3, the agent learned to transport the attached object towards the placement zone. This stage was harder than reaching and attaching because the agent had to maintain attachment, control height, and move towards a new goal at the same time.

**Table 6: Manual evaluation success rate. The trial count is set to 20 to match the recorded percentages.**

| Phase             | Episodes | Successes | Rate |
|-------------------|----------|-----------|------|
| Phase 1 Reach     | 20       | 19        | 95%  |
| Phase 2 Attach    | 20       | 19        | 95%  |
| Phase 3 Transport | 20       | 17        | 85%  |
| Phase 4 Release   | 20       | 18        | 90%  |

Training failures included shaking, dragging the object, or moving in a direction that caused the arm to become stuck.

In Phase 4, the agent was trained to lower and release the object near the placement zone. The final behaviour improved compared with early attempts, but release remained one of the most delicate parts of the task. Common issues included early release, imperfect centring, and occasional object bouncing after release.

### 5.3 Success Rate

The final behaviour was evaluated manually in the Unity simulation. A trial was counted as successful when the agent completed the target behaviour of the corresponding phase within the maximum episode length.

The transport phase had the lowest success rate because it required the agent to coordinate multiple constraints: maintaining attachment, lifting the object, moving to the placement zone, and avoiding unstable joint movement. The release phase achieved a higher final success rate after reward adjustments, but it still remained sensitive to timing and object height.

### 5.4 Generalisation Evaluation

The evaluation was carried out with the same randomised workspace logic used during training. The target object and the safe placement zone were not fixed to a single coordinate; instead, they were reset within constrained ranges. Therefore, the success-rate test evaluates in-distribution generalisation across different object and placement-zone positions rather than memorisation of one fixed layout. The policy was able to handle these randomised positions in most trials, especially in the Reach, Attach, and Release stages.

However, the project did not include a separate out-of-distribution test, such as placing the target or placement zone far outside the training range. This remains a limitation of the evaluation. A stronger generalisation test would compare success rates inside the training range, near the workspace boundary, and outside the original randomisation range.

### 5.5 Failure Cases

**Unstable arm movement.** In the transport and release stages, the arm sometimes produced visible shaking. This may have happened because the reward encouraged the agent to move, lift, and remain stable at the same time. During exploration, the policy sometimes selected aggressive joint changes that caused oscillation.

**Early release.** In early versions of Phase 4, the gripper sometimes released the object before the target had reached a safe height. This made the object appear to be thrown into the placement zone rather than placed. This was improved by increasing the reward for lowering and by making safe release conditions stricter.

**Object not fully inside the zone.** In some trials, the object was transported near the placement zone but was not centred accurately. This shows that the agent learned approximate transport earlier than precise placement. More precise reward shaping or longer training could further improve this behaviour.

## 6 Reflection

### 6.1 What Worked Well

The most effective design choice was the staged training strategy. Splitting the task into smaller phases made each learning objective clearer. After Phase 1 learned reaching, Phase 2 could focus on attachment. After Phase 2 learned attachment, Phase 3 could focus on transport. Finally, Phase 4 could specialise in lowering and release. This made the task easier to debug than a single end-to-end setup.

The simplified magnetic attachment mechanism also improved training stability. Pure physical grasping in Unity can be sensitive to collision settings, joint shaking, friction, and contact forces. By using magnetic attachment, the project reduced grasping instability and allowed the reinforcement learning task to focus on arm movement and placement.

### 6.2 Limitations

The project still has several limitations. First, the gripper and attachment process were partly scripted rather than fully learned by the agent, so the grasping behaviour is not physically realistic. Second, the agent used direct position information from Unity rather than visual input, so the system is a simulation-control task rather than a complete perception-based robotic system. Third, although the robot could complete the main Pick-and-Place sequence, the release phase remained sensitive to height, timing, and object stability. Finally, the safe snap release in Step 4 increased evaluation stability, but it also simplified the final object physics after release.

### 6.3 Creative Practice and Media-Arts Integration

Although the main focus of this project was technical training, the same Pick-and-Place pipeline could also be used in creative robotics and media-arts contexts. For example, a robotic arm could be trained to arrange small objects as part of an installation, move symbolic objects in response to audience interaction, or perform slow repeated placement gestures as a kinetic sculpture. In this type of work, the aim would not only be industrial efficiency, but also expressive movement, timing, and interaction design. The staged RL pipeline developed here could provide a controllable foundation for these creative systems, while future versions could add sound, camera input, or audience-triggered target positions.

### 6.4 Future Work

Future work could improve the system in several ways. First, the magnetic attachment mechanism could be replaced with more realistic physical grasping. Second, the target and placement zone randomisation could be expanded to test generalisation. Third, visual observations from a camera could be introduced so that the agent learns from images rather than direct coordinate information. Fourth, the reward function could be refined to make release

smoother and more physically realistic. Finally, the same training structure could be tested with other robotic arms such as UR5, Panda, or Kinova.

## 6.5 Personal Reflection

This project showed that training a robotic arm with reinforcement learning is not only about selecting an algorithm. It also requires careful design of the environment, action range, observation space, reset logic, and reward function. Many failures were caused not by PPO itself, but by task design issues such as sparse rewards, unsafe physical settings, or overly difficult stage objectives. Through this project, I gained a clearer understanding of both the potential and the limitations of reinforcement learning for robotic control.

## 7 Conclusion

This project implemented a staged reinforcement learning approach for a simulated robotic Pick-and-Place task in Unity ML-Agents. By dividing the task into Reach, Attach, Transport, and Lower-and-Release phases, the agent was able to build complex manipulation behaviour from simpler sub-skills. The training curves and manual success-rate evaluation indicate that curriculum-style training was

effective for this task. However, the system still depends on simplified grasping and direct position observations, and further work is needed to improve realism, robustness, and generalisation.

## References

- [1] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press, Cambridge, MA, USA.
- [2] Jens Kober, J. Andrew Bagnell, and Jan Peters. 2013. Reinforcement Learning in Robotics: A Survey. *The International Journal of Robotics Research* 32, 11 (2013), 1238–1274. <https://doi.org/10.1177/0278364913495721>
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*. <https://arxiv.org/abs/1707.06347>
- [4] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. 2018. Unity: A General Platform for Intelligent Agents. *arXiv preprint arXiv:1809.02627*. <https://arxiv.org/abs/1809.02627>
- [5] Unity Technologies. 2026. Unity ML-Agents Toolkit. GitHub repository and documentation. <https://github.com/Unity-Technologies/ml-agents>. Accessed 23 June 2026.
- [6] Unity Technologies. 2026. Unity Robotics Hub: Pick-and-Place Tutorial. GitHub repository documentation. [https://github.com/Unity-Technologies/Unity-Robotics-Hub/tree/main/tutorials/pick\\_and\\_place](https://github.com/Unity-Technologies/Unity-Robotics-Hub/tree/main/tutorials/pick_and_place). Accessed 23 June 2026.
- [7] Unity Technologies. 2026. URDF Importer for Unity. GitHub repository documentation. <https://github.com/Unity-Technologies/URDF-Importer>. Accessed 23 June 2026.